

REPORTE DE PROYECTO

Gestión de turnos de enfermeras en un hospital

Análisis y Diseño / Programación Orientada a Objetos

Grupo	23
Integrantes	Dillan Fernando Orellana Cepeda Sebastián Ignacio Rojas Rivera Yaril Germán Salinas Retamales
Versión del informe	Revisión posterior al feedback EP1–EP12
Alcance	Aplicación de gestión de enfermeras y turnos con consola, Swing y persistencia CSV
Enfoque	Diseño orientado a objetos, separación de responsabilidades y explicación funcional

Contenido

1. Resumen ejecutivo
2. Respuesta al feedback EP1–EP12
3. SIA-1: Análisis de datos y principales funcionalidades
4. SIA-2: Diseño conceptual y UML
5. SIA-3: Buenas prácticas, modularización y responsabilidades
6. SIA-4: Colecciones JCF y colección anidada
7. SIA-5: Sobrecarga de métodos
8. SIA-6: Herencia, sobreescritura y polimorfismo
9. SIA-7: Inserción y listado
10. SIA-8: Búsqueda, modificación y eliminación
11. SIA-9: Funcionalidades de utilidad de negocio
12. SIA-10: Interfaz de consola e interfaz gráfica
13. SIA-11: Persistencia CSV y procesamiento batch
14. SIA-12: Excepciones propias y manejo de errores
15. SIA-13: Uso de Git y GitHub
16. SIA-O4: Aplicación del patrón MVC
17. Validación funcional y resultados
18. Conclusiones

1. Resumen ejecutivo

El proyecto implementa una aplicación para administrar enfermeras y sus turnos de trabajo en un hospital. El sistema permite mantener datos de personal, asociar jornadas, consultar información y aplicar reglas de disponibilidad. La solución mantiene dos formas de interacción —consola y Swing— y una persistencia común en CSV, de modo que la forma de uso no modifica las reglas centrales del sistema.

Se centró en tres problemas recurrentes: la explicación; el diseño; y responsabilidades. Por ello se separa explícitamente vista, controlador, servicio, modelo y persistencia, incorpora un contrato de repositorio para desacoplar el almacenamiento y reemplaza la sobreescritura poco significativa de identificación por una operación de carga horaria que sí cambia su comportamiento según el tipo de trabajador.

El criterio utilizado para tomar estas decisiones fue mantener cada responsabilidad cerca del concepto que representa. Así, la enfermera administra sus turnos, el turno sabe determinar si está activo y calcular su duración, el servicio coordina reglas de negocio y la persistencia traduce objetos a registros CSV. Esta separación reduce el acoplamiento y hace que las decisiones del programa puedan explicarse desde el diseño, no sólo desde la implementación.

2. Respuesta al feedback EP1–EP12

La siguiente matriz resume cómo se abordó cada observación. La intención no es convertir el informe en una checklist, sino dejar explícito qué decisión de diseño se incorporó para responder al comentario y cómo se refleja posteriormente en la explicación.

EP	Observación	Respuesta incorporada
EP1	Análisis superficial	Se amplían las justificaciones de las decisiones: Map por RUT, List de turnos, servicio separado de persistencia, encapsulamiento, carga horaria y reglas de disponibilidad.
EP2	No había UML	Se incorporan un UML de dominio y un UML de arquitectura actualizados, coherentes con el código revisado.
EP3	Responsabilidades mezcladas	GestorHospital pasa al paquete servicio, PersistenciaCSV queda como almacenamiento, se crea RepositorioHospital y se incorpora ConversorTurnos.
EP4	Falta de diseño y explicación	El informe prioriza modelos, flujos y argumentos de diseño; se deja de usar el código como evidencia principal.
EP5	Sobrecarga sin análisis conceptual	Se explica el propósito común de cada sobrecarga y por qué las dos entradas representan la misma operación desde distintos puntos de partida.
EP6	Sobreescritura poco significativa	Trabajador define calcularCargaHoraria(); Enfermera calcula sus horas y Administrador representa una carga de 0 h en el alcance actual.
EP7	Explicación superficial y con código	Se incorporan capturas de interfaz para mostrar el comportamiento observable de las funcionalidades.
EP8	Falta explicar funcionalidades	Cada funcionalidad incluye objetivo, flujo, regla de negocio y resultado esperado/observado.

EP	Observación	Respuesta incorporada
EP9	Faltaban imágenes y análisis	Se agregan capturas para listado, turnos, búsquedas, disponibilidad y carga horaria.
EP10	Interfaz ventana incompleta	Se agrega el listado de todos los turnos y la consulta de carga horaria a la interfaz Swing; también se documentan las tres pestañas.
EP11	Código como evidencia de excepciones	Se explica el propósito de cada excepción y su recorrido desde servicio hasta controlador y vista, acompañado por un diagrama de flujo.
EP12	20 commits	Se conserva el historial de desarrollo registrado para la entrega.

3. SIA-1: Análisis de datos y principales funcionalidades

3.1. Problema y datos que debe representar el sistema

El problema consiste en administrar personal de enfermería y organizar jornadas de trabajo sin perder la relación entre una trabajadora y los turnos que le corresponden. Esto genera dos necesidades distintas: identificar rápidamente al personal y, una vez encontrada una enfermera, recorrer y modificar sus jornadas. La representación del dominio se diseñó alrededor de esa relación.

Trabajador concentra los datos comunes a los trabajadores del hospital: RUT y nombre. Enfermera agrega la especialidad y su lista de turnos. Administrador comparte la identidad básica, pero en el alcance del problema no administra jornadas. Turno contiene los datos de una jornada: identificador, fecha, tipo, estado, horario, sector y observaciones. Los valores de tipo y estado se expresan mediante enumeraciones para limitar la entrada a categorías válidas del dominio.

El RUT se usa como clave del mapa principal porque es el dato que distingue de forma natural a una enfermera. La lista de turnos se mantiene dentro de cada Enfermera porque un turno no se consulta en el sistema como una entidad independiente de su trabajadora: para llegar al turno primero se identifica a la enfermera y luego se busca dentro de sus jornadas. Esta decisión hace explícita la relación uno a muchos del dominio.

3.2. Funcionalidades principales y razón de su existencia

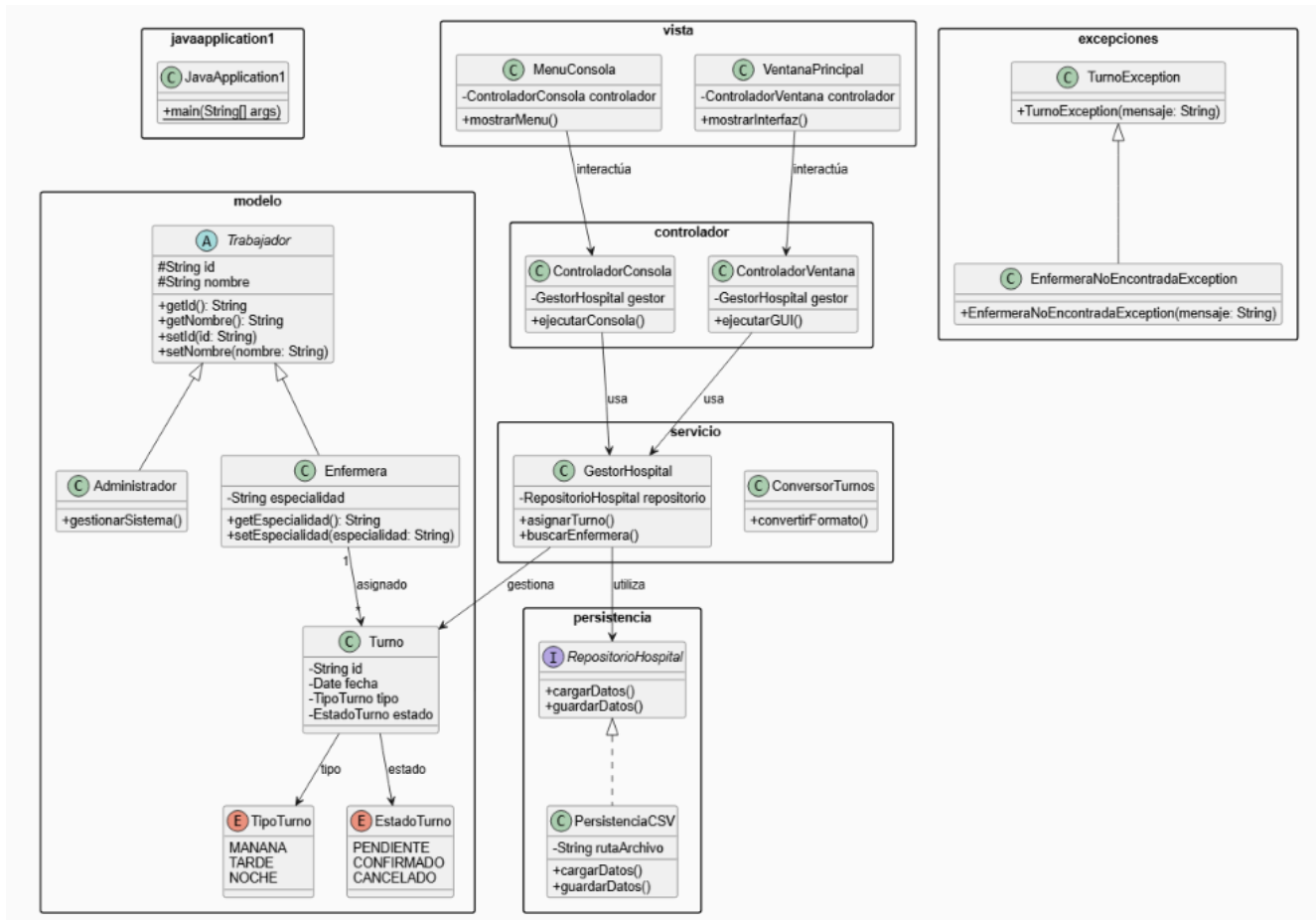
Las funcionalidades CRUD de enfermeras y turnos permiten mantener el estado del sistema. Sin embargo, el objetivo del programa no termina en registrar datos. Las consultas por especialidad y disponibilidad transforman la información almacenada en decisiones operativas: permiten detectar qué enfermeras cumplen un perfil y cuáles pueden ser asignadas a una fecha concreta.

La búsqueda de disponibilidad utiliza una regla deliberada: una enfermera no está disponible si tiene, para la fecha consultada, un turno cuyo estado sea distinto de CANCELADO. El estado cancelado se considera no bloqueante porque la jornada ya no representa una obligación activa. La regla queda encapsulada en Enfermera, donde tiene sentido semánticamente, y el servicio la reutiliza al recorrer el personal. Así se evita repetir la misma condición en la consola y en Swing.

Además se incorpora el cálculo de carga horaria. Esta funcionalidad permite recorrer los turnos activos de una enfermera, calcular la duración de cada jornada y acumular las horas. El cálculo considera también turnos nocturnos que terminan después de medianoche, por lo que un turno de 22:00 a 08:00 se interpreta como 10 horas y no como una duración negativa.

Decisión clave de análisis: La interfaz no define qué significa “estar disponible” ni cómo se calcula una jornada. Esas reglas pertenecen al dominio/servicio. Las interfaces sólo solicitan la operación y muestran su resultado.

4. SIA-2: Diseño conceptual y UML



4.1. UML de dominio

El UML de dominio representa las entidades que modelan el problema y las relaciones necesarias para operarlo. Trabajador se define como abstracta porque expresa información común y una operación cuyo comportamiento depende del tipo concreto. Enfermera y Administrador heredan esa base. Turno se mantiene como entidad separada porque una jornada tiene identidad y estado propios, mientras que TipoTurno y EstadoTurno restringen valores permitidos.

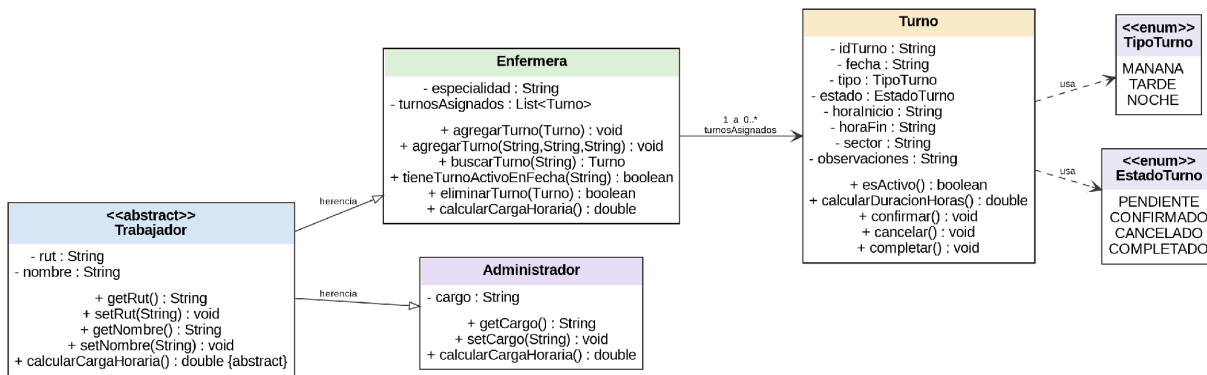


Figura 1. UML conceptual actualizado del dominio y sus relaciones.

La multiplicidad entre Enfermera y Turno es 1 a 0..*: una enfermera puede existir sin turnos o poseer varios. El sentido del vínculo justifica que la colección esté dentro de Enfermera. También se busca que el modelo proteja esa colección: la consulta de turnos devuelve una copia y las modificaciones se realizan mediante operaciones de la propia entidad, evitando que una vista pueda reemplazar directamente la lista interna.

4.2. UML de arquitectura

La arquitectura actual separa responsabilidades. JavaApplication1 inicia el sistema; las vistas interactúan con el usuario; los controladores adaptan las entradas y coordinan la solicitud; GestorHospital aplica las reglas; RepositorioHospital define el contrato de almacenamiento; PersistenciaCSV implementa ese contrato; y las excepciones representan errores propios del dominio.

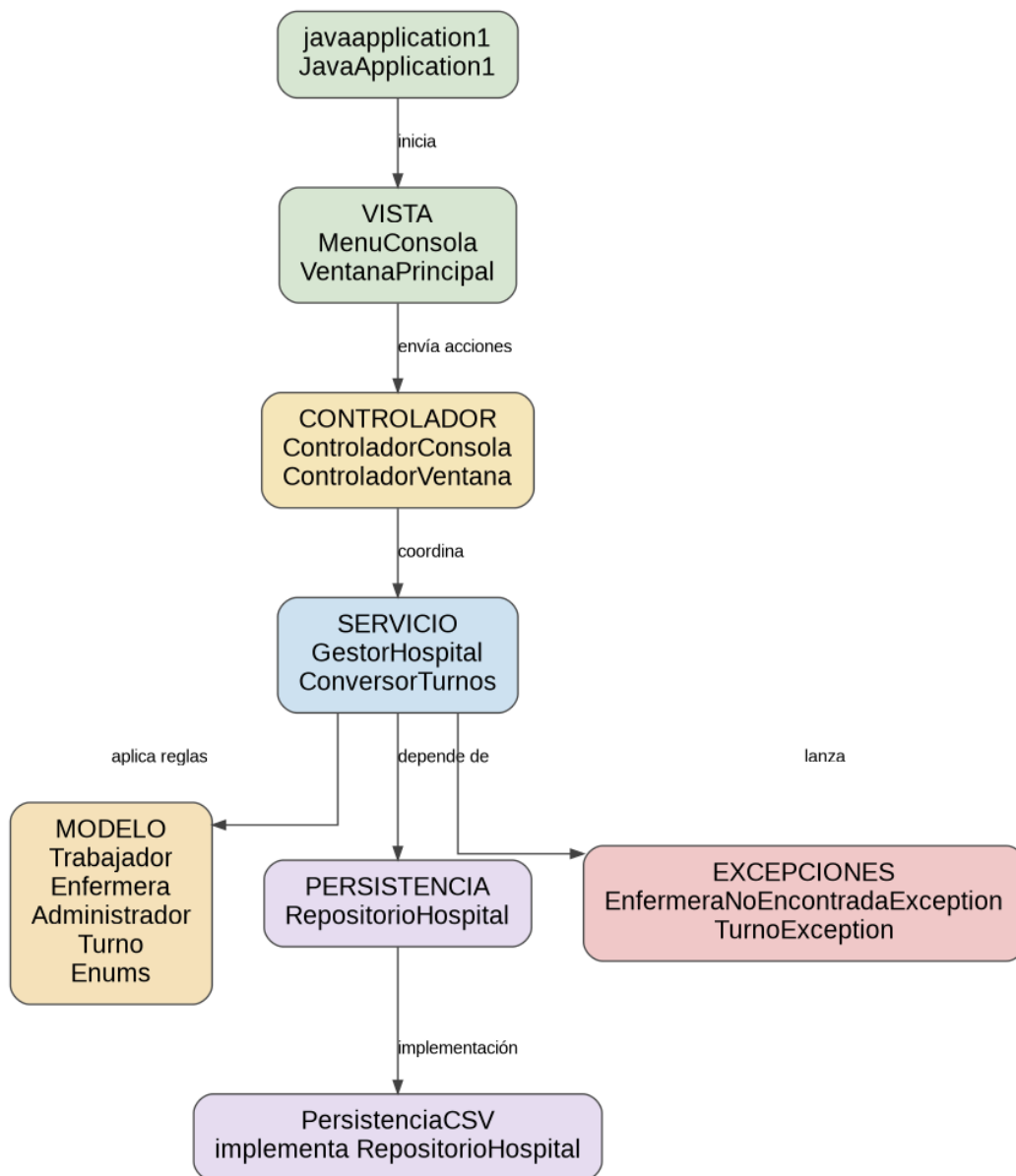


Figura 2. Arquitectura actual: vista, controladores, servicio, modelo, persistencia y excepciones.

La decisión más importante respecto del diseño es que GestorHospital no depende directamente de una clase CSV concreta. El servicio recibe un RepositorioHospital y, por defecto, se construye con PersistenciaCSV. Esto permite mantener el mismo comportamiento de negocio aunque en el futuro el almacenamiento sea otro. Además, facilita pruebas porque el servicio puede recibir un repositorio en memoria sin tocar archivos reales.

5. SIA-3: Buenas prácticas, modularización y responsabilidades

5.1. Separación por responsabilidades

La modularización se revisó siguiendo la idea de “una razón principal para cambiar”. MenuConsola y VentanaPrincipal cambian cuando cambia la forma de interacción; los controladores cambian cuando cambia la adaptación entre interfaz y servicio; GestorHospital cambia cuando cambian las reglas del

negocio; PersistenciaCSV cambia cuando cambia el mecanismo de almacenamiento; y las entidades cambian cuando cambia el modelo que representan.

La separación es especialmente relevante. La versión inicial conocía directamente el CSV. En la revisión, la persistencia queda detrás de RepositorioHospital y además se creó ConversorTurnos para concentrar la transformación de textos de la interfaz a TipoTurno y EstadoTurno. De esta forma, el servicio trabaja con conceptos del dominio y no con detalles de presentación o del formato del archivo.

5.2. Encapsulamiento y protección del estado

La colección de turnos es un punto sensible porque es un estado interno de Enfermera. Entregar directamente la lista permitiría que otra clase agregara o eliminara turnos sin pasar por las operaciones diseñadas para la entidad. Por eso getTurnosAsignados() devuelve una copia. El servicio, además, agrupa los turnos en estructuras que tampoco permiten modificaciones externas. La finalidad no es sólo “cumplir private”, sino mantener un estado que sólo pueda cambiar mediante operaciones conocidas.

5.3. Documentación interna y formato del código

Las clases principales y sus operaciones de dominio cuentan con documentación Javadoc orientada a explicar propósito, entradas, salidas y excepciones relevantes.

6. SIA-4: Colecciones JCF y colección anidada

La colección principal es Map<String, Enfermera>. El mapa permite localizar una enfermera por su RUT sin recorrer toda la colección para las operaciones de búsqueda y modificación. Además, LinkedHashMap mantiene un orden de inserción estable para que los listados sean reproducibles, lo que resulta útil en ambas interfaces.

Cada Enfermera contiene List<Turno>. La lista es adecuada porque una trabajadora puede tener una secuencia de varias jornadas y el programa necesita recorrerlas para mostrarlas, buscar un identificador, verificar disponibilidad y calcular carga horaria. La estructura anidada puede leerse como “RUT de enfermera -> persona -> lista de turnos” y coincide con la forma en que el usuario consulta el sistema.

Relación con el rendimiento: Las búsquedas de enfermera por RUT se benefician del acceso del Map, mientras que las consultas dentro de una enfermera recorren su lista de turnos. En el diseño actual esto mantiene el modelo simple y coherente con la estructura del problema.

7. SIA-5: Sobrecarga de métodos

7.1. Sobrecarga en GestorHospital

La operación de registrar una enfermera tiene dos entradas válidas. Una recibe un objeto Enfermera ya construido; la otra recibe RUT, nombre y especialidad y construye la entidad antes de registrarla. Las dos versiones tienen el mismo propósito y convergen en la misma validación de inserción. Esto evita duplicar reglas y hace que la sobrecarga sea una conveniencia de construcción, no dos comportamientos distintos.

7.2. Sobrecarga en Enfermera

La misma idea aparece al agregar un turno. La entidad puede recibir un Turno ya construido o los datos básicos de una jornada. En ambos casos la acción conceptual es idéntica: asociar una jornada a la enfermera. La segunda variante es útil cuando los datos provienen directamente de una interfaz; la primera resulta natural cuando otra parte del sistema ya construyó el objeto. El diseño mantiene una única

responsabilidad para la incorporación y evita que la vista tenga que conocer cómo se inicializa completamente un turno.

7.3. Razón del diseño

La sobrecarga se utiliza sólo cuando las variantes siguen representando la misma operación. No se crearon nombres distintos para acciones equivalentes ni se agregaron parámetros artificiales. La decisión busca mejorar la usabilidad de la API del dominio manteniendo un significado único por operación.

8. SIA-6: Herencia, sobreescritura y polimorfismo

La jerarquía Trabajador ahora define `calcularCargaHoraria()`. Enfermera redefine la operación acumulando la duración de sus turnos activos. Administrador también comparte la operación, pero devuelve 0 horas porque, dentro del alcance de este sistema, los administradores no tienen jornadas asignadas. El mismo mensaje —“calcular carga horaria”— puede enviarse a una referencia de tipo Trabajador y cada clase responde según su naturaleza.

La decisión tiene una ventaja conceptual: la operación se relaciona directamente con el problema que administra el sistema y no sólo con la forma de imprimir un objeto. Además la clase participa de una funcionalidad compartida de la jerarquía, aunque su resultado represente una situación distinta del de Enfermera.

Polimorfismo en el dominio: El significado común está en Trabajador; el detalle de cómo se calcula la carga pertenece a cada subclase. La vista no necesita saber qué clase concreta recibió para solicitar el cálculo.

9. SIA-7: Inserción y listado para ambas colecciones

La inserción y el listado se mantienen para las dos colecciones principales. En Enfermeras, el usuario registra el personal y posteriormente puede mostrar el conjunto existente. En Turnos, se asignan jornadas a una enfermera y luego se muestran agrupadas por la persona a la que pertenecen. En ambos casos el controlador solicita los datos al servicio y la vista se ocupa sólo de presentar el resultado.

Enfermeras | Turnos | Búsquedas

Gestión de Enfermeras

Datos de la Enfermera

RUT:

Nombre:

Especialidad:

Agregar / Modificar **Eliminar** **Mostrar Enfermeras**

Buscar Enfermera **Ver Carga Horaria**

--- ENFERMERAS REGISTRADAS ---

Enfermera [21231313] - ddadsa (da)

Enfermera [22222222-2] - Bárbara Silva (Pediatria)

Enfermera [11111111-1] - Ana Rojas (Urgencias)

Figura 3. Interfaz Swing: registro y listado de enfermeras. La pestaña incorpora también la consulta de carga horaria.

Enfermeras **Turnos** **Búsquedas**

Gestión de Turnos

Datos del Turno

RUT Enfermera:

ID Turno:

Fecha:

Tipo: **MANANA** ▼

Estado: **PENDIENTE** ▼

Asignar **Mostrar Turnos** **Buscar** **Modificar** **Eliminar**

--- TURNOS REGISTRADOS ---

Enfermera [22222222-2] - Bárbara Silva (Pediatria)
 Turno [T002] - Fecha: 11-09-2026 | Tipo: NOCHE | Estado: PENDIENTE | Horario: 22:00 - 08:00 | Sector: Pediatría | Observaciones: Turno nocturno

Enfermera [11111111-1] - Ana Rojas (Urgencias)

Figura 4. Interfaz Swing: listado de todos los turnos agrupados por enfermera.

La decisión de mostrar los turnos agrupados por enfermera mantiene visible la estructura anidada del dominio. En lugar de entregar una lista plana sin contexto, cada turno aparece bajo su responsable, lo que facilita la lectura y evita que el usuario tenga que reconstruir la relación manualmente.

10. SIA-8: Búsqueda, modificación y eliminación

Las operaciones de búsqueda, modificación y eliminación utilizan el RUT para identificar enfermeras. En el caso de los turnos, la operación usa el RUT más el identificador del turno, porque el identificador es único dentro de la colección de la enfermera y la jornada se administra como parte de esa relación. La regla reduce ambigüedades y refleja el modelo en lugar de inventar una identificación global que el problema no necesita.

En la interfaz gráfica, la búsqueda de una enfermera completa nuevamente los campos del formulario y muestra el resultado. En la gestión de turnos, buscar permite recuperar la jornada, mientras modificar y eliminar pasan por el controlador y el servicio. Cuando la entidad no existe, el flujo utiliza los mecanismos de error del dominio que se describen en SIA-12.

The screenshot shows a web application interface titled "Gestión de Enfermeras". At the top, there are three tabs: "Enfermeras", "Turnos", and "Búsquedas". The "Enfermeras" tab is active. Below the tabs, the main content area is titled "Gestión de Enfermeras". Inside this area, there is a section labeled "Datos de la Enfermera" which contains three input fields: "RUT:" with the value "11111111-1", "Nombre:", and "Especialidad:". Below these fields, there are five buttons arranged in two rows: "Agregar / Modificar", "Eliminar", and "Mostrar Enfermeras" in the first row; "Buscar Enfermera" and "Ver Carga Horaria" in the second row. Below the main content area, there is a section titled "--- CARGA HORARIA ---" which displays the text "RUT: 11111111-1" and "Horas de turnos activos: 8.0".

Figura 5. Consulta de carga horaria desde la interfaz de enfermeras.

11. SIA-9: Funcionalidades de utilidad de negocio

11.1. Búsqueda por especialidad

Esta consulta responde a una necesidad operativa distinta del CRUD: encontrar personal por perfil profesional. La comparación ignora mayúsculas y minúsculas para evitar que la forma de escribir la especialidad sea tratada como una diferencia del dominio.

Enfermeras Turnos **Búsquedas**

Consultas del Sistema

Buscar por Especialidad

Buscar Enfermeras Disponibles

--- RESULTADOS POR ESPECIALIDAD ---

Enfermera [11111111-1] - Ana Rojas (Urgencias)

Figura 6. Resultado de búsqueda de enfermeras por especialidad.

11.2. Consulta de disponibilidad

La disponibilidad combina dos condiciones: coincidencia de especialidad y ausencia de un turno activo en la fecha solicitada. Esta lógica se diseñó como una consulta de negocio porque transforma varias piezas de datos en una respuesta que ayuda a planificar una asignación. La fecha se consulta sobre los turnos de cada enfermera y los estados CANCELADO, CONFIRMADO, PENDIENTE y COMPLETADO se interpretan según la regla definida para disponibilidad.

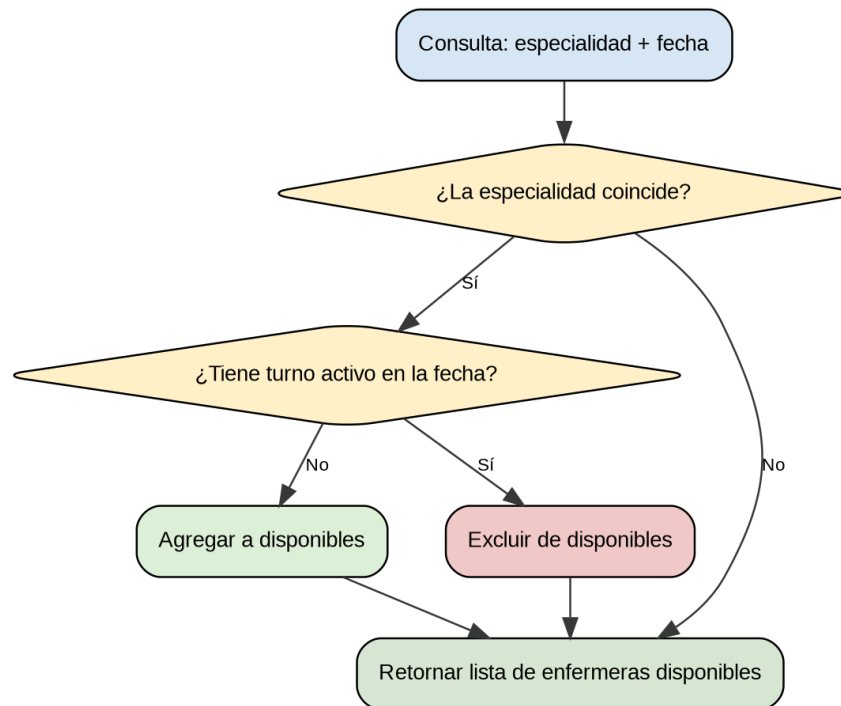


Figura 7. Flujo conceptual para determinar qué enfermeras están disponibles.

Enfermeras

Turnos

Búsquedas

Consultas del Sistema

Buscar por Especialidad

Buscar Enfermeras Disponibles

--- ENFERMERAS DISPONIBLES ---

Enfermera [11111111-1] - Ana Rojas {Urgencias}

Figura 8. Resultado de la consulta de enfermeras disponibles.

11.3. Carga horaria

La carga horaria complementa las consultas anteriores porque no busca una persona, sino que resume la cantidad de horas activas que tiene asignadas. Turno calcula la duración y Enfermera acumula esos valores. Esta división evita que la vista tenga que interpretar horarios y permite reutilizar la misma regla desde consola y Swing.

12. SIA-10: Interfaz de consola e interfaz gráfica

12.1. Interfaz gráfica Swing

La ventana principal organiza la interacción en tres pestañas: Enfermeras, Turnos y Búsquedas. Esto separa visualmente las operaciones de administración de personal, la administración de jornadas y las consultas. La pestaña Enfermeras contiene registro/modificación, eliminación, listado, búsqueda y carga horaria; Turnos contiene asignación, listado total, búsqueda, modificación y eliminación; Búsquedas ofrece las consultas por especialidad y disponibilidad.

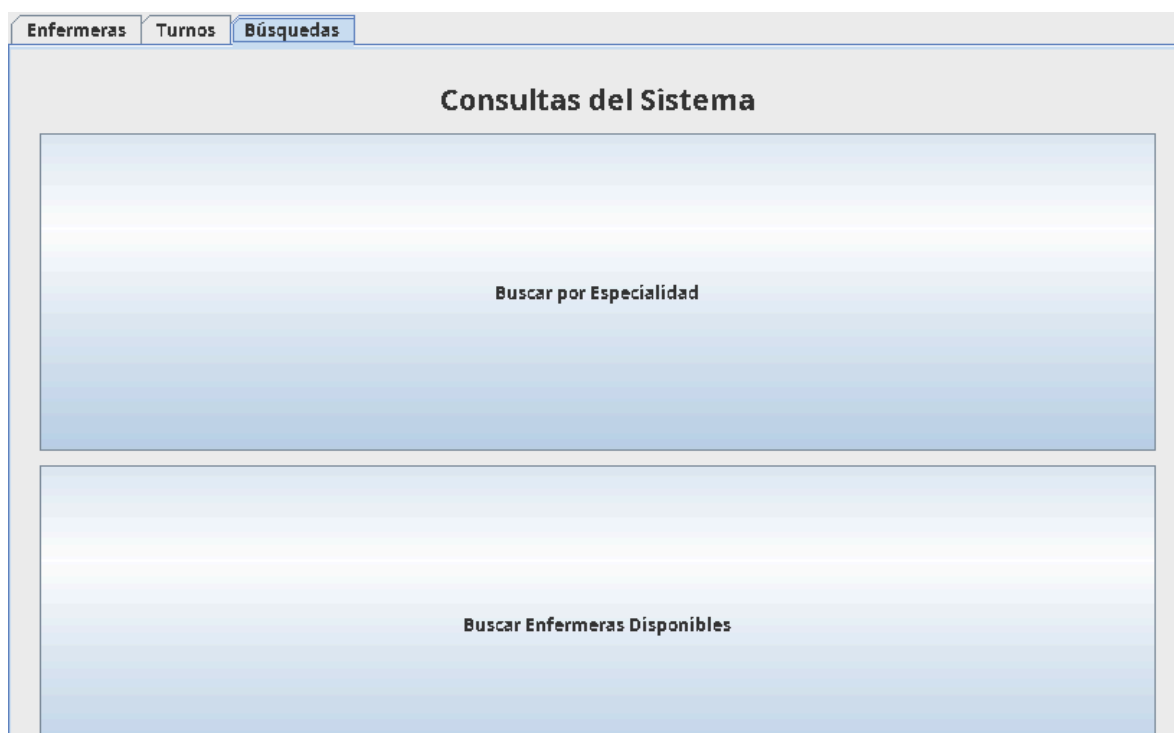


Figura 9. Pestaña Búsquedas de la interfaz gráfica.

La versión revisada incorpora específicamente el listado de todos los turnos y la consulta de carga horaria en la ventana, respondiendo al problema señalado en el feedback. La interfaz sigue reutilizando el mismo servicio que consola, de modo que añadir una función visual no implica duplicar las reglas de negocio.

12.2. Interfaz de consola

La consola ofrece el mismo conjunto de operaciones mediante un menú numérico. La opción de carga horaria se incorporó también aquí para mantener paridad funcional con Swing. La diferencia entre ambas interfaces es la forma de interacción, no el conjunto de reglas que gobiernan el sistema.

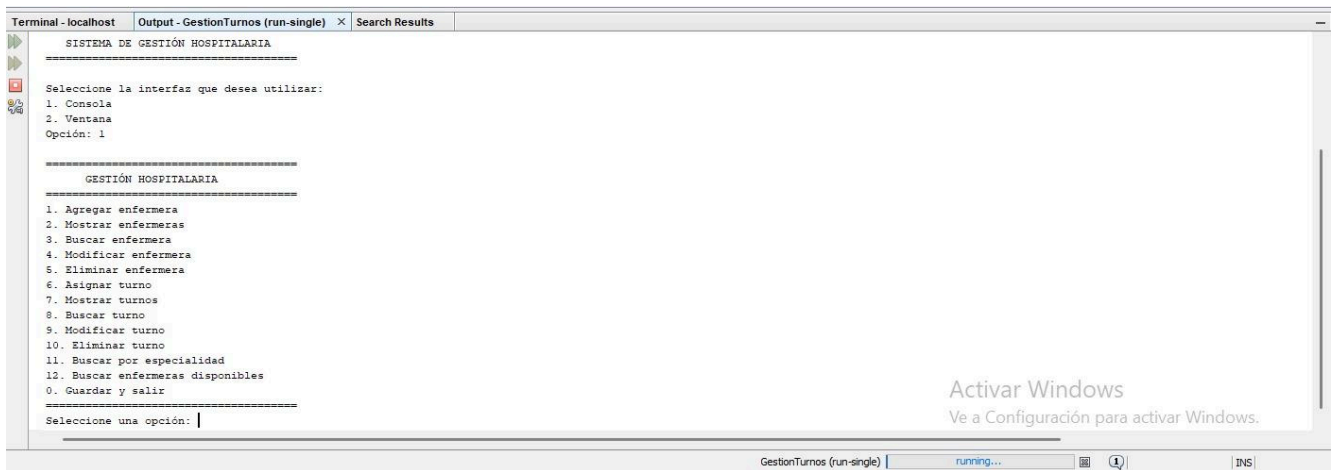


Figura 10. Menú de consola con las funcionalidades principales y la nueva consulta de carga horaria.

Criterio de diseño de las dos interfaces: El usuario puede cambiar de modalidad sin cambiar el modelo de datos ni las reglas de negocio. Los controladores funcionan como adaptadores específicos para cada forma de interacción.

13. SIA-11: Persistencia CSV y procesamiento batch

La persistencia se separó mediante la interfaz `RepositorioHospital`. El servicio trabaja contra ese contrato y `PersistenciaCSV` se encarga de convertir los objetos a registros CSV y reconstruirlos al cargar. La razón de esta separación es que el formato de almacenamiento es una decisión técnica, mientras que registrar una enfermera o consultar disponibilidad son reglas del problema.

El sistema opera con un esquema batch sencillo: al iniciar se carga el conjunto de datos desde el CSV; durante la ejecución se trabaja en memoria; y al terminar se guarda el estado acumulado. En consola, el guardado ocurre al seleccionar salir. En Swing se solicita al cerrar la aplicación mediante un hook de apagado. Esto evita reescribir el archivo después de cada clic y concentra la serialización en un único componente.

La revisión también eliminó mensajes de interfaz desde `PersistenciaCSV`. Un componente de almacenamiento ya no decide cómo comunicar un error al usuario; devuelve el resultado o la excepción necesaria para que el controlador determine cómo presentarlo. Esta decisión es importante para mantener la independencia entre persistencia y presentación.

14. SIA-12: Excepciones propias y manejo de errores

14.1. Por qué existen excepciones de dominio

En este proyecto hay situaciones que no representan un error técnico del lenguaje, sino una operación inválida dentro del problema. Solicitar un turno para una enfermera inexistente y manipular un turno que no existe son ejemplos de esas condiciones. Por eso se definieron `EnfermeraNoEncontradaException` y `TurnoException`. Las clases permiten comunicar claramente qué tipo de situación ocurrió sin mezclarla con errores de interfaz.

14.2. Recorrido del error

GestorHospital detecta la condición y lanza la excepción correspondiente. El controlador la captura con try-catch y conserva el mensaje en una variable de estado. La vista consulta ese mensaje y lo presenta mediante texto o diálogo según la interfaz. El servicio, por tanto, no sabe si el usuario está mirando una consola o una ventana Swing.

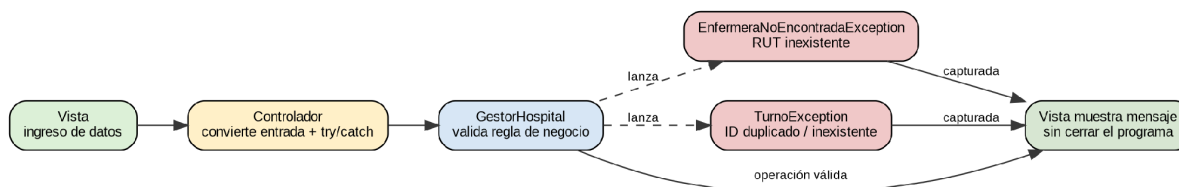


Figura 11. Flujo de una excepción de dominio desde la regla de negocio hasta la interfaz.

La separación es importante porque las excepciones no se crearon sólo para “mostrar un catch”, sino para representar condiciones de negocio reconocibles. El beneficio es que la misma regla puede producir un manejo equivalente en las dos interfaces y evitar cierres inesperados del programa ante entradas inválidas.

Validación complementaria: La interfaz gráfica también verifica entradas vacías antes de solicitar la operación. La validación de formato de negocio y la existencia de entidades quedan en capas diferentes para no cargar toda la responsabilidad en la vista.

15. SIA-13: Uso de Git y GitHub

El proyecto utiliza Git como historial incremental de desarrollo y GitHub como repositorio remoto. Los cambios se organizaron alrededor de etapas del modelo, persistencia, controladores, vistas y encapsulamiento, lo que permite asociar modificaciones del proyecto con decisiones concretas de diseño.

Repositorio remoto: github.com/sebastianrojasr02-spec/proyecto-turnos-enfermeras-en-un-hospital

Para la evaluación se conserva el historial de 20 commits señalado en el feedback. El valor del historial no está sólo en la cantidad, sino en que muestra que el proyecto fue evolucionando de forma incremental y no como una única entrega final.

16. SIA-O4: Aplicación del patrón MVC

La solución aplica una separación basada en Modelo-Vista-Controlador, complementada por un servicio de dominio y persistencia. El modelo contiene las entidades y enums; la vista contiene MenuConsola y VentanaPrincipal; los controladores reciben acciones de ambas interfaces; GestorHospital concentra las reglas; y RepositorioHospital/PersistenciaCSV se encargan del almacenamiento.

La elección responde al problema práctico de tener dos interfaces sobre la misma lógica. Si la consola y Swing tuvieran implementadas sus propias reglas, cada cambio tendría que repetirse y existiría riesgo de resultados distintos. Al compartir el servicio, ambas interfaces solicitan las mismas operaciones y difieren solamente en cómo recogen y presentan la información.

ConversorTurnos complementa este esquema porque concentra la traducción entre los textos seleccionados en la interfaz y los enums del modelo. De esta forma, la conversión no se replica en cada controlador ni se mezcla con las reglas de negocio.

17. Conclusiones

La relación Enfermera–Turno, el Map principal y la colección anidada responden al problema; las sobrecargas mantienen un propósito único; y la herencia expresa una funcionalidad real mediante el cálculo polimórfico de carga horaria.

En arquitectura, la separación entre vista, controladores, servicio y persistencia evita que una decisión local contamine el resto del sistema. RepositorioHospital desacopla el negocio del CSV, mientras ConversorTurnos concentra una conversión técnica que de otro modo podría repetirse en las interfaces. El encapsulamiento de turnos protege el estado interno y las excepciones de dominio hacen explícitas las condiciones inválidas del sistema.

Desde el punto de vista funcional, la versión actual mantiene las operaciones CRUD, las consultas de especialidad y disponibilidad y añade la carga horaria como funcionalidad de negocio y como ejemplo de polimorfismo. La interfaz Swing incorpora también el listado de todos los turnos y conserva paridad con la consola en las operaciones principales.

Idea central del diseño: El sistema está organizado para que cada capa responda a una pregunta distinta: ¿qué representa el dato?, ¿qué regla se debe aplicar?, ¿cómo se adapta la interacción? y ¿cómo se almacena? Esa separación es la base de la mantenibilidad del proyecto.